

Лабораторна робота №2

Тема: Списки, кортежі, словники і множини у мові Python.

Мета: Набуття практичних навичок створення програм, що використовують списки, кортежі, словники і множини у мові Python.

Лабораторні завдання

Завдання 1. Скласти програму, яка містить функцію, що заповнює список з n елементів випадковими цілими числами з інтервалу від a до b .

Опис коду розробленої програми

Програму реалізовано за допомогою модуля random. Вона зчитує параметри списку, перевіряє їх на коректність (з автоматичною корекцією меж інтервалу) і генерує список випадкових чисел через окрему функцію. Помилки некоректного вводу обробляються блоком try...except.

Приклади роботи програми:

```
PS D:\УНІВЕРСИТЕТ32\СШ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШ\Liulkova/L2/2_1.py
Введіть кількість елементів списку (n): 25
Введіть початок інтервалу (a): 4
Введіть кінець інтервалу (b): 36
Згенерований список: [13, 6, 21, 6, 25, 11, 35, 17, 6, 14, 36, 26, 36, 36, 22, 19, 22, 27, 22, 33, 10, 24, 7, 25, 28]
PS D:\УНІВЕРСИТЕТ32\СШ\Liulkova>
```

Рис. 1.1 Успішна генерація списку за коректно введеними даними

```
PS D:\УНІВЕРСИТЕТ32\СШ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШ\Liulkova/L2/2_1.py
Введіть кількість елементів списку (n): 40
Введіть початок інтервалу (a): 100
Введіть кінець інтервалу (b): 75
Помилка: a більше за b. Міняємо їх місцями для коректної роботи
Згенерований список: [80, 85, 83, 90, 94, 99, 82, 90, 94, 81, 77, 75, 87, 99, 84, 99, 76, 86, 96, 97, 93, 89, 86, 95, 81, 85, 75, 85, 96, 76, 86, 87, 93, 99, 76, 77, 79, 88, 81, 77]
PS D:\УНІВЕРСИТЕТ32\СШ\Liulkova>
```

Рис. 1.2 Виконання програми при введенні $a > b$ (демонстрація автоматичного обміну меж інтервалу)

```
PS D:\УНІВЕРСИТЕТ32\СШ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШ\Liulkova/L2/2_1.py
Введіть кількість елементів списку (n): -30
Введіть початок інтервалу (a): 7
Введіть кінець інтервалу (b): 80
Помилка: кількість елементів має бути більшою за нуль
PS D:\УНІВЕРСИТЕТ32\СШ\Liulkova>
```

Рис. 1.3 Перехоплення помилок (введення від'ємної кількості елементів або нечислових значень)

Завдання 2. Дано список a з n чисел, серед яких можливі від'ємні. Скласти програму, яка містить функцію, що записує всі від'ємні числа списку a в другий список. Розробити два варіанти програми:

- 1) без використання генератора списку;
- 2) з використанням генератора списку.

Опис коду розробленої програми

Програму реалізовано у двох варіантах: відбір від'ємних чисел за допомогою класичного циклу `for` з методом `append()` та за допомогою компактної конструкції генератора списків (`list comprehension`). Початковий список формується випадковими числами через модуль `random`. Коректність введення кількості елементів контролюється блоком `try...except`

Приклад роботи програми:

```
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L2/2_2.py
Введіть числа списку через пробіл (наприклад: 5 -3 12 -7 0): -8 7 0 4 -6
Початковий список: [-8, 7, 0, 4, -6]
Варіант 1 (цикл): [-8, -6]
Варіант 2 (генератор): [-8, -6]
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>
```

Рис. 2.1 Успішне виконання програми (виведення згенерованого списку та результатів роботи обох функцій)

Завдання 3. Дан список, в якому можливо є задане число *n*. Скласти програму, яка містить функцію, що дозволяє отримати новий список, в якому будуть всі елементи заданого списку, крім числа *n*. Розробити два варіанти програми:

- 1) без використання генератора списку;
- 2) з використанням генератора списку.

Опис коду розробленої програми

Програму реалізовано у двох варіантах: видалення заданого числа *n* зі списку за допомогою класичного циклу `for` та за допомогою генератора списків. Початковий список формується з введеного користувачем рядка з використанням методу `split()`. Захист від введення некоректних даних (наприклад, літер замість чисел) забезпечується блоком `try...except`.

Приклади роботи програми:

```
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L2/2_3.py
Введіть числа списку через пробіл: 1 45 3 8 6 6
Введіть число n, яке потрібно видалити: 45
Початковий список: [1, 45, 3, 8, 6, 6]
Варіант 1 (цикл): [1, 3, 8, 6, 6]
Варіант 2 (генератор): [1, 3, 8, 6, 6]
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>
```

Рис. 3.1 Успішне виконання програми (коректне видалення існуючого числа *n* з початкового списку обома методами)

```
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L2/2_3.py
Введіть числа списку через пробіл: 1 4 6 3 7 8
Введіть число n, яке потрібно видалити: 10
Початковий список: [1, 4, 6, 3, 7, 8]
Варіант 1 (цикл): [1, 4, 6, 3, 7, 8]
Варіант 2 (генератор): [1, 4, 6, 3, 7, 8]
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>
```

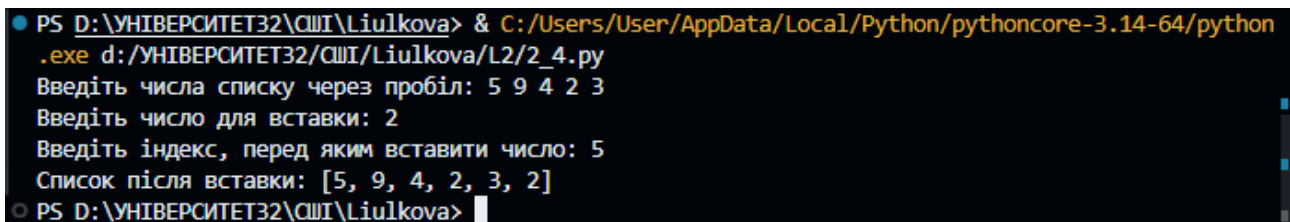
Рис. 3.2 Виконання програми у випадку, коли задане число n відсутнє у списку (результуючі списки залишаються ідентичними початковому)

Завдання 4. Скласти програму, яка містить функцію вставки в список заданого числа перед елементом із заданим індексом без використання методу `insert()`.

Опис коду розробленої програми

Програму реалізовано за допомогою користувацької функції, яка виконує вставку нового елемента через зрізи списків (slicing) та їх конкатенацію (+), що дозволяє уникнути використання вбудованого методу `insert()`. Вхідні дані користувача перетворюються на список чисел. Коректність введення (захист від нечислових символів) забезпечується блоком `try...except`.

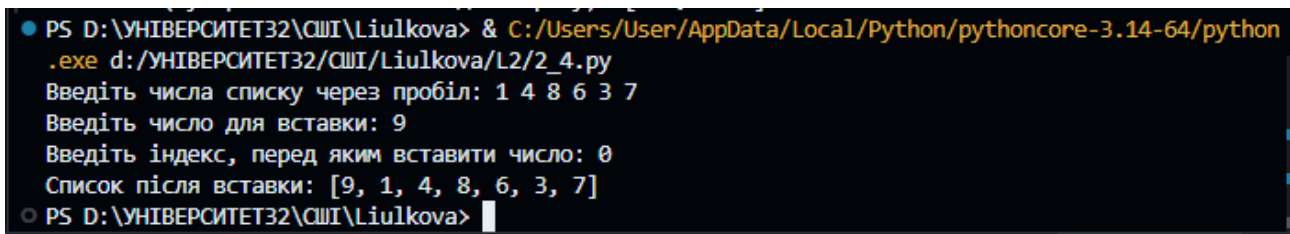
Приклади роботи програми:



```

PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python
.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L2/2_4.py
Введіть числа списку через пробіл: 5 9 4 2 3
Введіть число для вставки: 2
Введіть індекс, перед яким вставити число: 5
Список після вставки: [5, 9, 4, 2, 3, 2]
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>
    
```

Рис. 4.1 Успішне виконання програми (коректна вставка заданого числа всередину списку за вказаним індексом)



```

PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python
.exe d:/УНІВЕРСИТЕТ32/СШІ/Liulkova/L2/2_4.py
Введіть числа списку через пробіл: 1 4 8 6 3 7
Введіть число для вставки: 9
Введіть індекс, перед яким вставити число: 0
Список після вставки: [9, 1, 4, 8, 6, 3, 7]
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>
    
```

Рис. 4.2 Вставка елемента за граничними індексами (наприклад, індекс 0 для вставки на початок або індекс, що перевищує довжину списку, для вставки в кінець)

Завдання 5. Скласти програму, в якій використовується словник з даними про студентські групи такого вигляду: {<група>: {<кількість студентів>, <ПІБ старости>}} Програма повинна містити меню, пункти якого відповідають наступним функціям:

- 1) повідомлення про кількість студентів у зазначеній групі;
- 2) повідомлення ПІБ старости у зазначеній групі;
- 3) створення кортежу всіх груп, в яких кількість студентів не перевищує заданого значення;
- 4) створення кортежу всіх груп, в яких кількість студентів не менше заданого значення;

- 5) зміна кількості студентів у зазначеній групі;
- 6) зміна ПІБ старости у зазначеній групі;
- 7) додавання нової групи в словник;
- 8) видалення заданої групи зі словника;
- 9) отримання множини ПІБ старост зазначених груп;
- 10) вихід з програми. Результати виконання реалізованих зазначених функцій, що відповідають пунктам меню, повинні відображатися на екрані.

Опис коду розробленої програми

Програму реалізовано на базі вкладки словника, де ключами виступають назви студентських груп, а значеннями — внутрішні словники з даними про кількість студентів та ПІБ старости. Взаємодія з користувачем організована через інтерактивне текстове меню, яке працює у нескінченному циклі `while True`. Маршрутизація вибору користувача обробляється розгалуженою конструкцією `if-elif-else`. Для реалізації аналітичних запитів (пошук груп за кількістю студентів або вибірка старост) ефективно застосовуються генератори кортежів та множин у поєднанні з методом словника `.items()`. Програма повноцінно підтримує модифікацію структури: оновлення наявних значень за ключем, додавання нових груп та видалення записів за допомогою оператора `del`. Для запобігання збоєм при введенні числових даних (наприклад, при оновленні кількості студентів) використовується обробка винятків блоком `try...except ValueError`.

Приклади роботи програми:

```
PS D:\УНІВЕРСИТЕТ32\ОШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe d:/УНІВЕРСИТЕТ32\ОШІ\Liulkova/L2/2_5.py

=====
Поточний стан словника: {'КС-21': {'students': 30, 'head': 'Іванов І.І.'}, 'КІ-22': {'students': 10, 'head': 'Сидоренко С.С.'}, 'КН-23': {'students': 23, 'head': 'Петренко П.П.'}}
=====
1) Кількість студентів у зазначеній групі
2) ПІБ старости у зазначеній групі
3) Кортеж груп (студентів <= заданого значення)
4) Кортеж груп (студентів >= заданого значення)
5) Зміна кількості студентів у групі
6) Зміна ПІБ старости у групі
7) Додавання нової групи
8) Видалення заданої групи
9) Множина ПІБ старост зазначених груп
10) Вихід з програми
=====
Оберіть пункт меню (1-10):
```

Рис. 5.1 Виклик меню та виведення кількості студентів у заданій групі (пункт 1)

```
=====
Оберіть пункт меню (1-10): 2
Введіть назву групи: КН-23
Староста: Петренко П.П.
```

Рис. 5.2 Виведення ПІБ старости заданої групи (пункт 2)

```
=====
Оберіть пункт меню (1-10): 3
Введіть максимальну кількість студентів: 38
Результат: ('КС-21', 'КІ-22', 'КН-23')
```

Рис. 5.3 Створення кортежу груп за максимальною кількістю студентів (пункт 3)

```
=====
Оберіть пункт меню (1-10): 4
Введіть мінімальну кількість студентів: 10
Результат: ('КС-21', 'КІ-22', 'КН-23')
```

Рис. 5.4 Створення кортежу груп за мінімальною кількістю студентів (пункт 4)

```
=====
Оберіть пункт меню (1-10): 5
Введіть назву групи: КІ-22
Введіть нову кількість студентів: 15
Дані оновлено
```

Рис. 5.5 Оновлення кількості студентів у наявній групі (пункт 5)

```
=====
Оберіть пункт меню (1-10): 6
Введіть назву групи: КС-21
Введіть нове ПІБ старости: Ів
Дані оновлено
```

Рис. 5.6 Зміна ПІБ старости у наявній групі (пункт 6)

```
=====
Оберіть пункт меню (1-10): 1
Введіть назву групи: КІ-21
Такої групи не знайдено
```

Рис. 5.7 Коректне завершення роботи програми (пункт 10) та/або демонстрація обробки невірнього вводу

Завдання 6. Катя і Іван грають у гру за наступними правилами. Іван «загадує» натуральне число від 1 до n і повідомляє Каті це максимально можливе число n . Катя намагається відгадати «загадане» число, запитуючи Івана про деяку послідовність чисел. На кожне з питань Каті хитрий Іван вибирає такий варіант відповіді *Так* або *Ні*, щоб послідовність можливих задуманих чисел залишалася якомога більшою. Наприклад, якщо Іван задумав число від 1 до 7, а Катя запитала про числа 2 і 3, то Іван відповість *Ні*, а якщо Катя запитає про 1, 2, 3, 5 то Іван відповість *Так*. Якщо ж Катя в своєму питанні перераховує рівно половину з задуманих чисел поточної послідовності, то Іван

завжди відповідає *Ні*. Іван при відповіді враховує всі попередні питання Каті і свої відповіді на них, тобто множина можливих задуманих чисел не збільшується. Але Катя не завжди запам'ятовує і враховує відповіді Івана на свої питання.

Отже, дано послідовність питань Каті. Програма повинна наводити відповіді Івана на них. Перший рядок вхідних даних містить число *n* – найбільше число, яке міг загадати Іван. Далі повинні йти рядки, які містять питання Каті, у вигляді набору чисел, розділених пропусками, і відповіді Івана. Останній рядок вхідних даних повинен містити одне слово *Все*, після якого програма повинна вивести в порядку зростання, через пропуск всі числа, які міг би загадати Іван після відповіді на всі питання Каті.

Наприклад, нехай загадане число не перевищує 10. Тоді хід роботи програми може бути таким:

```
10
3 4 5 6 8 9
Так
1 2 4 5
Ні
1 3 5 6 7
Ні
Все
8 9
```

Опис коду розробленої програми

Основою алгоритму є використання структури даних "множина" (set), що ідеально підходить для швидких математичних операцій над унікальними елементами. На початку генерується базова множина всіх можливих чисел від 1 до введеного максимального значення *n*. У циклі `while` програма зчитує запитання (перетворюючи рядок введених чисел на множину) і паралельно розраховує два можливі сценарії:

1. Якщо відповідь "Так", залишаються лише ті числа, які є спільними для початкової множини та запитання (використовується метод `intersection`).
2. Якщо відповідь "Ні", залишаються числа з початкової множини, яких немає у запитанні (використовується метод `difference`).

Для реалізації "хитрості" Івана програма порівнює розміри цих двох нових множин за допомогою функції `len()` і перезаписує поточну множину більшим варіантом, виводячи відповідну відповідь на екран (якщо вони рівні, перевага надається відповіді "Ні"). Ігровий цикл переривається при введенні ключового слова "Все", після чого залишкові можливі числа сортується вбудованою

функцією `sorted()` та виводяться через пробіл. На всіх етапах введення даних (як для `n`, так і для запитань) передбачено захист від некоректного тексту за допомогою `try...except`.

Приклади роботи програми:

```
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python
.exe d:/УНІВЕРСИТЕТ32\СШІ\Liulkova/L2/2_6.py
Введіть максимальне число n:
10
Вводьте питання Каті (числа через пробіл) або слово 'Все' для завершення:
3 4 5 6 8 9
Так
1 2 4 5
Ні
1 3 5 6 7
Ні
все
Результат:
8 9
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova>
```

Завдання 7. При виконанні цього завдання всі вхідні дані повинні вводитись за допомогою клавіатури під час роботи програми:

13) дано список рядків. Скласти програму, яка з даного списку формує два списки за наступним правилом: елементами першого списку є ті елементи даного списку, які зустрічаються в ньому лише один раз, а елементами другого – всі ті елементи даного списку, які зустрічаються в ньому більше одного разу;

Опис коду розробленої програми

Програму реалізовано у вигляді функції, яка зчитує введений користувачем текст і розбиває його на список слів методом `split()`. У циклі `for` кожен елемент перевіряється за допомогою методу `count()`. Якщо елемент зустрічається у списку лише один раз, він додається до першого списку. Якщо більше одного разу — він потрапляє до другого списку (при цьому додаткова умова `if item not in list2` гарантує, що у другий список слово запишеться без дублювання).

Приклади роботи програми:

```
PS D:\УНІВЕРСИТЕТ32\СШІ\Liulkova> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python
.exe d:/УНІВЕРСИТЕТ32\СШІ\Liulkova/L2/2_7_13.py
Введіть список рядків (або слів) через пробіл: 1 5 7 93 5 4 93 вона 6 3 Вона
Початковий список: ['1', '5', '7', '93', '5', '4', '93', 'вона', '6', '3', 'Вона']
Список 1 (зустрічаються лише один раз): ['1', '7', '4', 'вона', '6', '3', 'Вона']
Список 2 (зустрічаються більше одного разу): ['5', '93']
```

Рис. 7.1 Стандартне виконання (введено як унікальні слова, так і ті, що повторюються)

Висновки. Під час виконання лабораторної роботи успішно закріплено знання та набуто практичних навичок роботи з базовими структурами даних

Python. Розроблено програми, що демонструють ефективну обробку списків за допомогою циклів, генераторів та зрізів. На практиці засвоєно управління вкладеними словниками через створення інтерактивного меню, а також застосування множин для швидких логіко-математичних операцій (перетин, різниця). Додатково закріплено використання незмінних кортежів для збереження відфільтрованих результатів. В усіх розроблених алгоритмах забезпечено надійність та стійкість коду до некоректного введення даних за допомогою блоків обробки винятків try...except.